从零？不，借用

牛津 102 花卉细粒度分类的迁移学习实验

55.46% → 92.96%

ResNet-18 · Oxford-102 · 2026

计算机眼里的图片

$$x \in \mathbb{R}^{3 \times 224 \times 224}$$

一张 224×224 的彩色图 = 150,528 个 0–255 的数字

- 3 个颜色通道：红、绿、蓝
- 模型看到的不是花，而是数字矩阵

神经网络在做什么

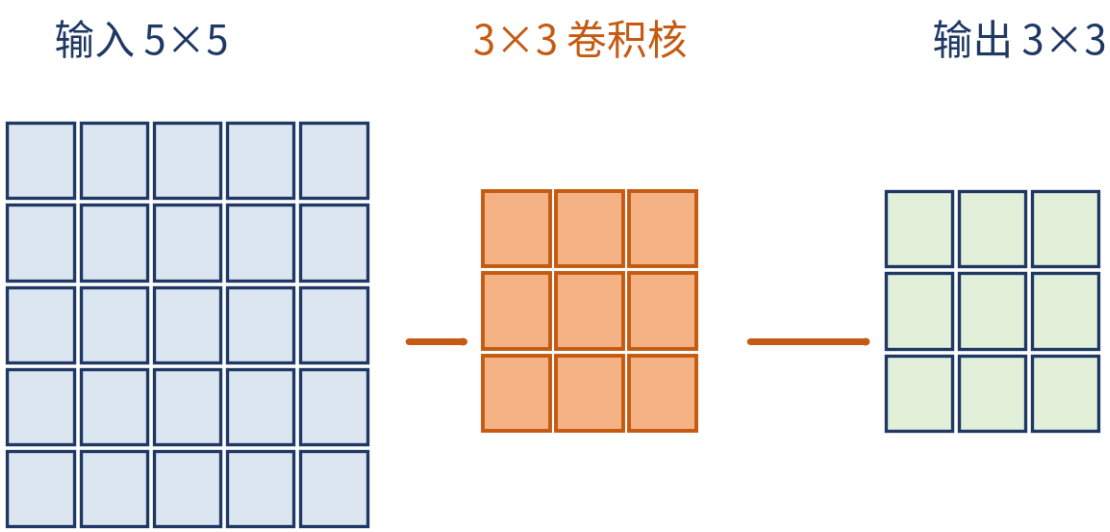
$$p_c = \frac{\exp(z_c)}{\sum_k \exp(z_k)}$$

102 个分数经过 softmax，变成 102 个概率

- z_c 越大，第 c 类概率越高
- 所有类别概率之和等于 1

卷积：一个小窗口扫过图片

$$y_{i,j} = f\left(\sum_{c,k,l} w_{c,k,l} \, x_{c,i+k,j+l} + b \right)$$



滑动窗口，共享权重

- 3×3 窗口在图上滑动
- 同一组权重反复使用
- 每次算加权和再激活

怎么知道模型错了

$$L = -\sum_{c} y_c \log p_c$$

正确类概率越低，罚得越狠

- 训练目标：让正确类概率接近 1
- 交叉熵越小，模型越自信且正确

怎么改：梯度下降

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

沿让损失变小的方向，挪一小步

- η 是步长：太大震荡，太小太慢
- 重复：算损失 → 求梯度 → 更新

为什么不能从零学

scratch

55.46%

120 epoch 从零训练

frozen

81.27%

12 epoch 只训分类头

预训练让模型从“记住训练集”转向可迁移特征

迁移学习：借别人学过的「看」的能力

$$\theta_{\text{backbone}} \leftarrow \theta^{*}_{\text{ImageNet}}, \quad \theta_{\text{head}} \leftarrow \text{random}$$

预训练： ImageNet 120 万张先学会看边缘、纹理、形状

- 骨干借过来；分类头重新随机初始化
- 先在通用视觉上“学会看”，再学认花

只换最后一层

$$52{,}326 = 512 \times 102 + 102$$

总参数	首阶段可训	可训占比
11,228,838	**52,326**	**0.47%**
ResNet-18	frozen / two-stage / strong	只换 512→102 分类头

骨干保留 ImageNet 权重；分类头把 512 维映射到 102 类

代码：换掉最后一层

```
model.fc = nn.Linear(model.fc.in_features, NUM_CLASSES)
return model
```

- `model.fc` 只替换分类头
- 512 维 → 102 类：骨干权重保持不变

输出层是唯一和 102 类绑定的部分；骨干不需要动

52,326 = 512×102 + 102 （回扣第 9 页）

两阶段微调

$$\eta_{\text{head}} = 10^{-3},\ \eta_{\text{backbone}} = 10^{-4}$$



- 解冻时重建优化器与 scheduler
- 解冻处允许 loss 抖动，不改语义

代码：冻结与解冻

```python
def set_backbone_trainable(model, trainable):
    for name, param in model.named_parameters():
        if name.startswith("fc."):
            param.requires_grad = True
        else:
            param.requires_grad = bool(trainable)
```

- fc 层始终 requires_grad=True
- 解冻后重建 optimizer，动量重置

数据增强 + 代码

原图



裁剪



翻转



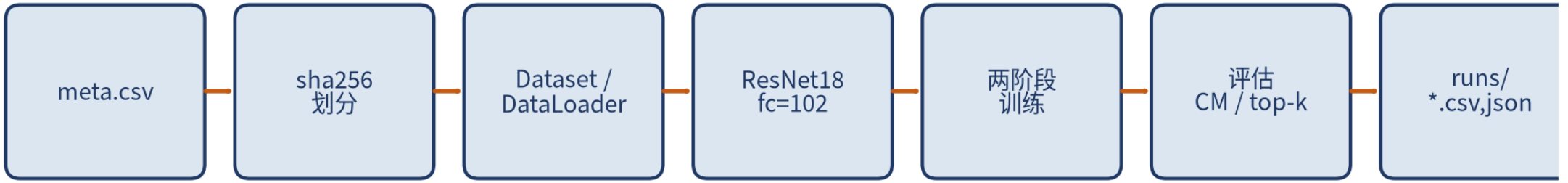
旋转



```python
AUG_OPS = {
    "baseline": [
        "RandomResizedCrop(224,scale=(0.6,1.0))",
        "RandomHorizontalFlip()",
    ],
    "strong": [
        "RandomResizedCrop(224,scale=(0.35,1.0))",
        "RandomHorizontalFlip()",
        "RandomRotation(25)",
        "ColorJitter(0.3,0.3,0.3,0.05)",
        "RandomErasing(p=0.25,scale=(0.02,0.15))",
    ],
}
```

- 裁剪 / 翻转 / 旋转 / 颜色抖动：一张图变多样本
- `AUG_OPS`： baseline 与 strong 两套配置

流水线：从数据到产物



数据 → 划分 → 模型 → 训练 → 评估 → 产物

```python
class Flowers(Dataset):
    def __init__(self, img_dir, rows, tf):
        self.img_dir = Path(img_dir)
        self.rows = list(rows)
        self.tf = tf

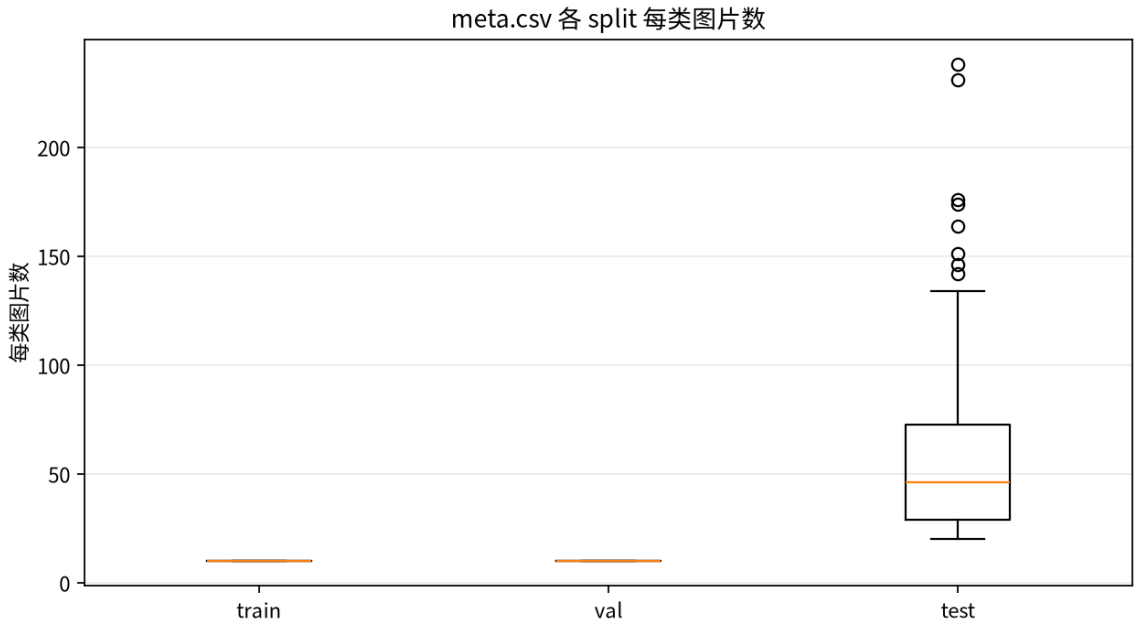
    def __len__(self):
        return len(self.rows)

    def __getitem__(self, index):
```

Dataset 负责把 image_id 和标签读成模型输入

数据集与划分

- 8189 张 / 102 类 / 每类 40–258
- 官方 split： 1020 / 1020 / 6149
- 我们的 train/val： 1734 / 306
- test 6149 只做最终评估



test 每类 20–238，分布不均

代码： sha256 分桶

```
def _hash_key(split_seed, image_id):
    return hashlib.sha256(f"{split_seed}:{image_id}".encode(
        "ascii")).hexdigest()


def build_split(meta_csv, split_seed=SPLIT_SEED,
    val_per_class=VAL_PER_CLASS):
    """纯函数：按 hash 分桶生成 {'train': [...], 'val': [...],
        'test': [...]}。"""
    by_class, test_rows = _read_meta(meta_csv)
    train_rows = []
```

- 每个 image_id 算 sha256 键
- 按 (k, image_id) 排序：前 3 张进 val

四组对照实验

scratch

不借：随机初始化，全部参数可训

frozen

借骨干：只训 52,326 个参数

two-stage

先借后放开： 5 轮冻结 + 20 轮解冻

strong

两阶段 + 强增强 + LS 0.1 + 35 轮

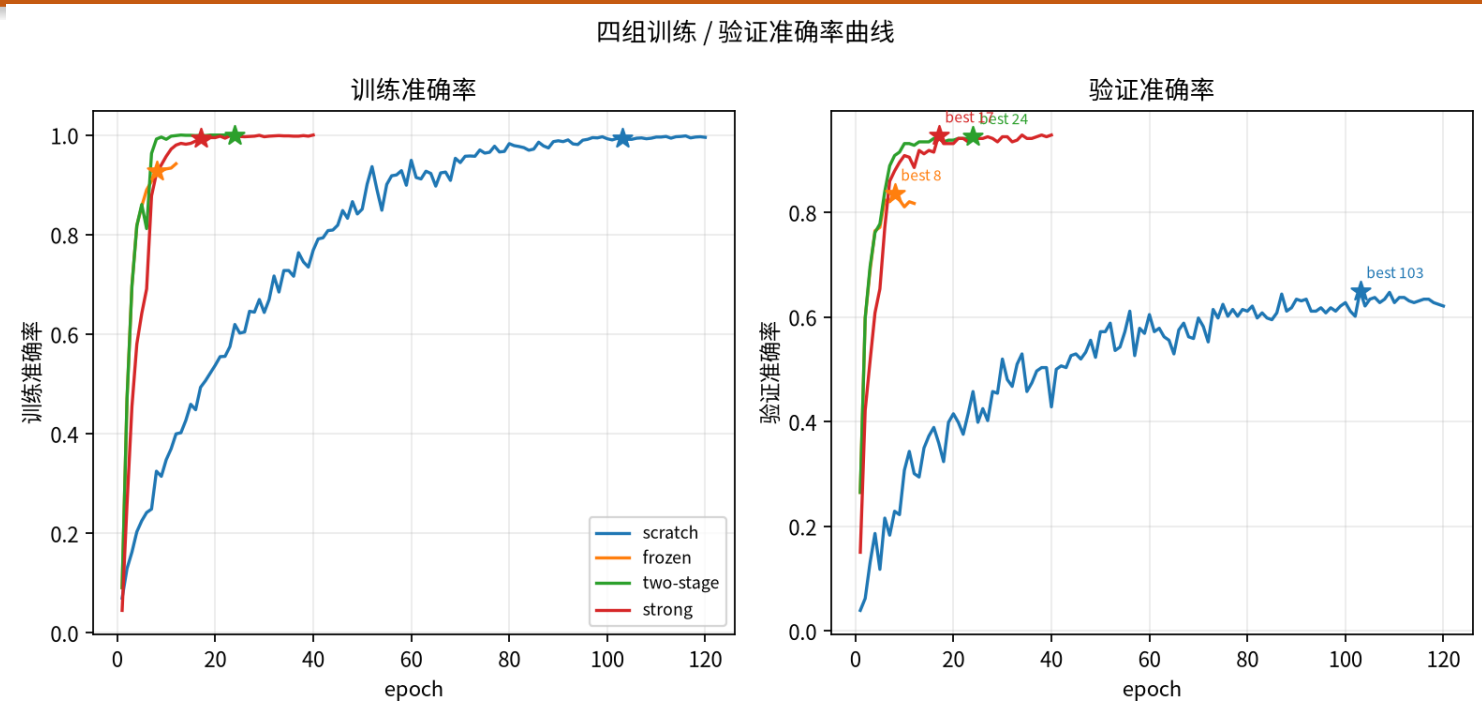
四组只差在“借多少”：从全随机到全解冻

结果总表

组别	top-1	top-3	macro-F1
scratch	55.46%	72.47%	54.80%
frozen	81.27%	92.11%	80.45%
two-stage	92.50%	97.69%	92.14%
strong	92.96%	97.76%	92.52%

全部为 test 集；数据源： report/results.csv

训练曲线与日志



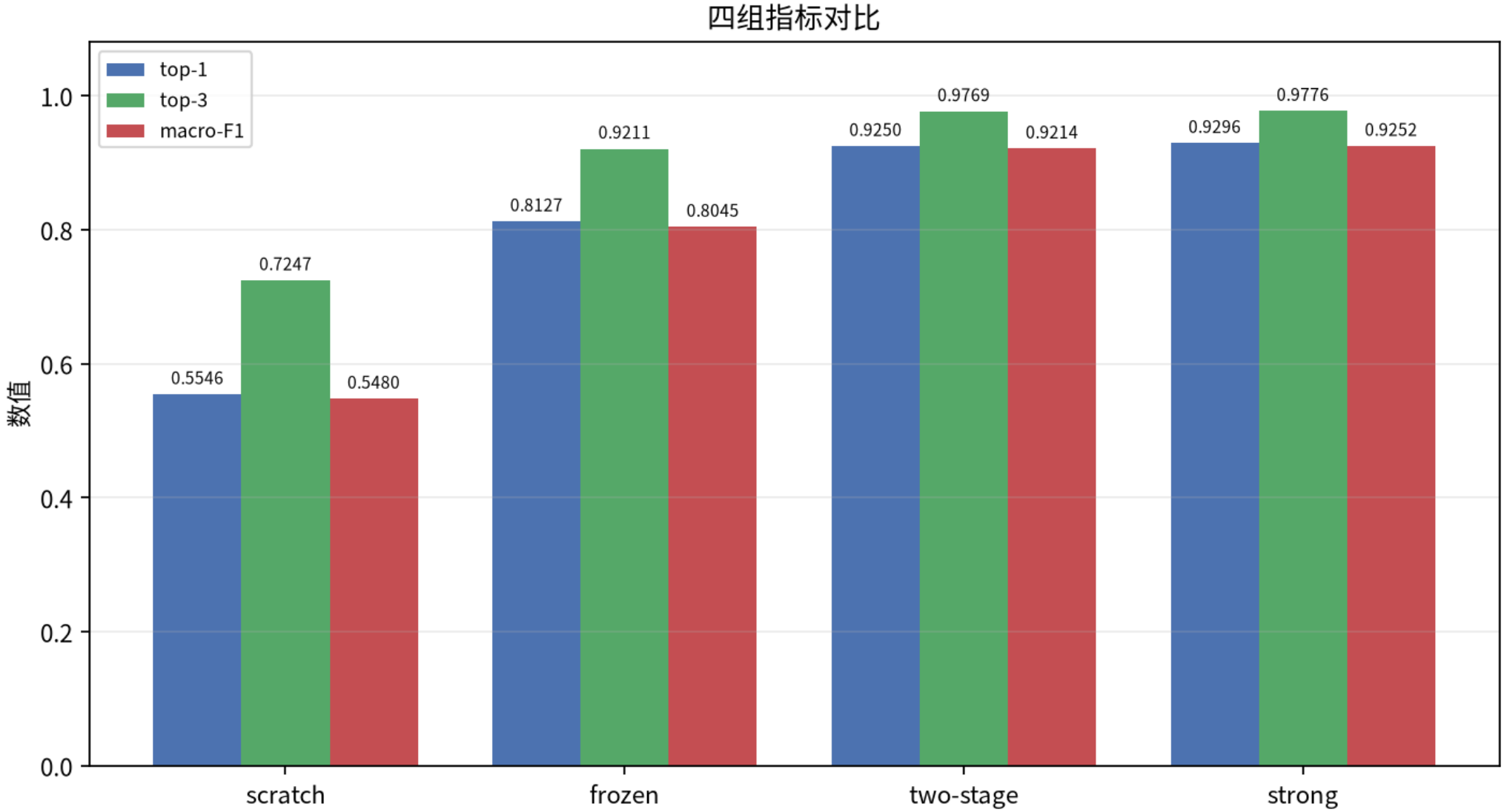
关键列： epoch / train_loss / val_loss / val_acc / phase

第 6 轮 = 解冻

```
epoch,train_loss,val_loss,val_acc,phase
1,4.354282,3.681311,0.264706,frozen
2,3.200952,2.781540,0.598039,frozen
3,2.400029,2.193030,0.696078,frozen
6,0.992110,0.668700,0.839869,unfrozen
```

runs/two-stage/history.csv： 第 1–3 、 6 轮

指标对比

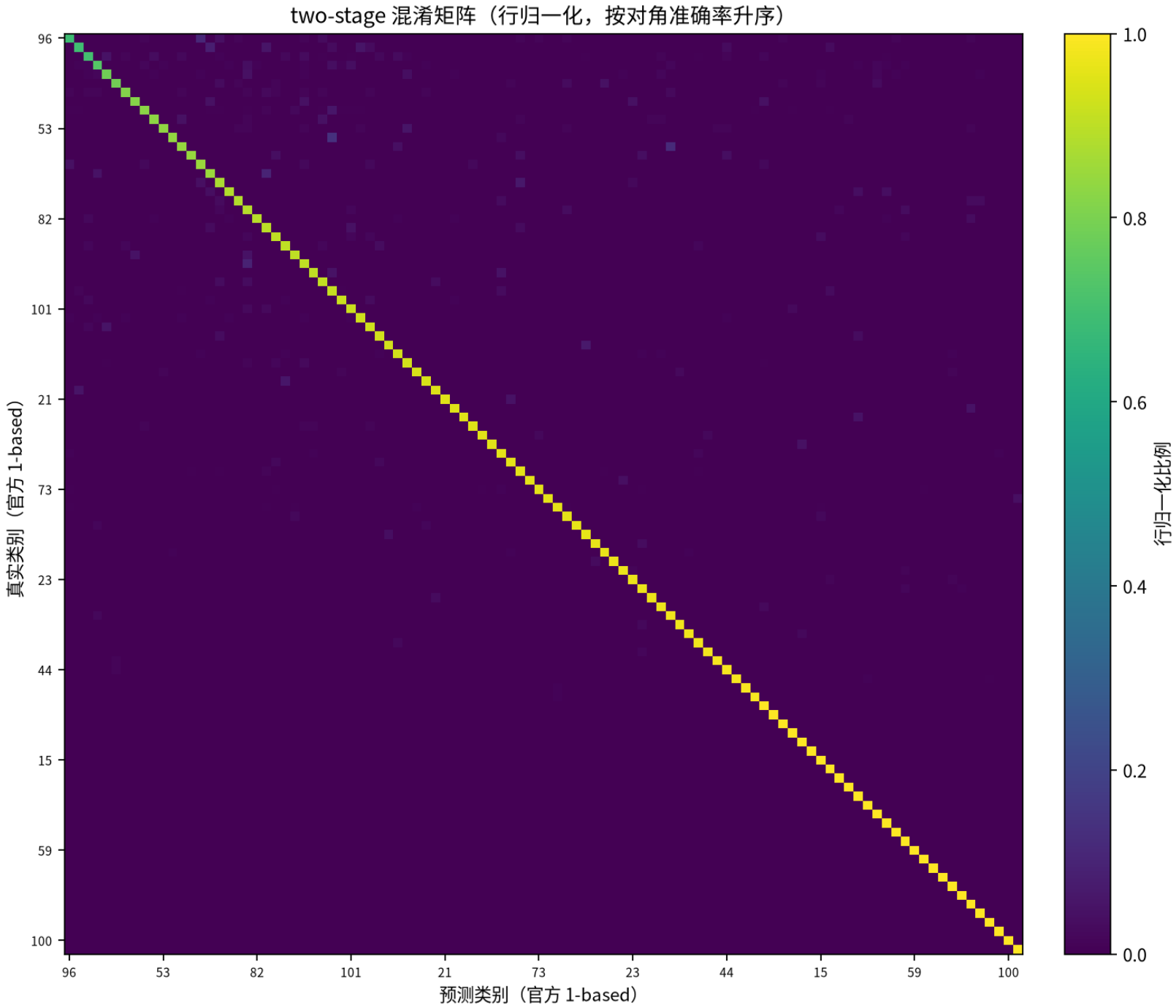


strong 92.96%， two-stage 92.50%，相差 0.46 个百分点

怎么看混淆矩阵

two-stage
102×102
行 = 真实
列 = 预测
对角线越亮越好

左上角最差：
camellia 69.01%



代码：评估与混淆矩阵

```python
top = probs.topk(5, dim=1).indices
preds = top[:, 0]
batch = labels.numel()

loss_sum += float(loss) * batch
total += batch
correct_top1 += int((preds == labels).sum().item())
correct_top3 += int((top[:, :3] ==
    labels.unsqueeze(1)).any(dim=1).sum().item())
correct_top5 += int((top[:, :5] ==
    labels.unsqueeze(1)).any(dim=1).sum().item())

flat_index = labels * num_classes + preds
cm.view(-1).index_add_(
    0, flat_index.cpu(), torch.ones(batch, dtype=torch.long)
)
true_probs = probs.gather(1, labels.unsqueeze(1)).squeeze(1)
truep_sum.index_add_(0, labels.cpu(), true_probs.double().cpu())
```

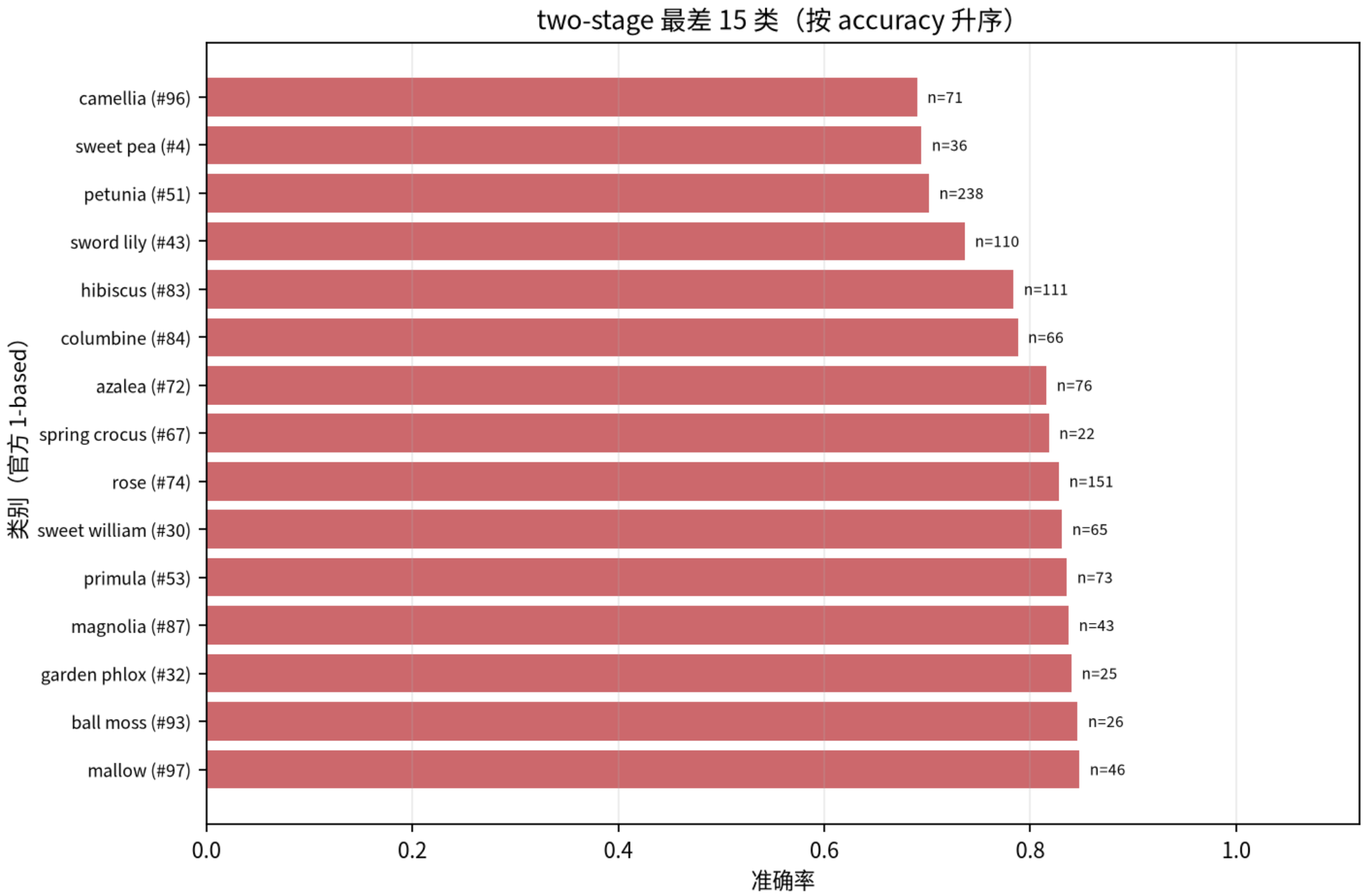
- topk(5) 一次算 top-1/3/5
- CM 用 index_add_ 逐 batch 累加

典型错例



true_prob 最小的 9 条中， camellia→mallow 有 4 条

最差类别



petunia
n=238
拖累最大

公开结果量级

方法	top-1	说明
Nilsback 2008	72.80%	103 类旧库 · 手工特征 +SVM
CCT-14/7×2	99.76%	from scratch · Transformer
ViT-L/16 (2026)	99.37%	303.4M · 官方划分
本实验 strong	92.96%	ResNet-18 · 11.2M

协议不同，不可直接比较；这里只做量级参照

结论、边界与踩坑

结论

- 预训练是主力： frozen 12 轮 81.27%
- 解冻跃迁： strong 92.96%
- scratch 仅 55.46%： 从零不划算

边界

- 单 seed，无种子方差
- 验证 306 张，每类 3 张
- strong 是三因素 bundle，不做单因素归因

踩坑记录

- `/kaggle/src` 只读 → 写 `/kaggle/working`
- `truep_sum` 102 vs 64 → 用 index_add_
- GPU 长队列 → CPU 冒烟先跑通

Q & A